SIMATS ENGINEERING

C PROGRAMMING

ASSIGNMENT REPORT

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Name: Anushka G

Register Number: 1912260668

Department: Electronics and Communication Engineering (ECE)

Assignment Type: Individual

Submission Date: 28-10-2026

Slot:C

Github link: https://github.com/anushkapearl21-jpg/co-3.git

Note: The supplied screenshots do not show the official tariff table and the final lines of Task 4. This report follows the visible task headings and requirements. The rates used in the demonstration are placeholders, not verified official rates.

Assignment Overview

Assignment title: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative.

Nature: Individual | Duration: 1 Week | Total: 50 marks

The assignment instructions require individual execution of every task, source code and input files, output screenshots, a README.md, progressive Git commits, and a working GitHub repository link.

Task 1 – Implement and Test the Billing Program (4 Marks)

Bloom's Level: K3 – Apply

Objective: Implement the billing program and test ordinary bills, slab boundaries, zero units, meter rollover, invalid input, and a small set of bills that can be calculated by hand.

Design and implementation

The program offers legacy meter mode and smart-meter mode. Legacy mode validates both readings in the 0–65535 range, calculates units, and calculates the progressive slab charge. If the current reading is lower than the previous reading, the user must explicitly confirm a rollover; otherwise the program rejects the reading. This makes the assignment's "lower reading without roll-over" case testable.

Illustrative slab rates used because the official rates are not visible in the screenshots:

Slab	Rate	Charge
0–100 units	Rs. 1.50/unit	units × 150 paise
101–200 units	Rs. 2.50/unit	first 100 at Rs. 1.50 + remainder at Rs. 2.50
Above 200 units	Rs. 4.00/unit	first 100 + next 100 + remainder at Rs. 4.00

The program stores charges as integer paise to avoid floating-point rounding in the bill calculation. The illustrative tariff values are defined as named constants so they can be changed in one place.

Test table: expected and actual results

Test	Input	Expected	Actual	Status
Exactly 100 units	Previous 0, current 100	100 units; Rs. 150.00	100 units; Rs. 150.00	Pass
Exactly 200 units	Previous 0, current 200	200 units; Rs. 400.00	200 units; Rs. 400.00	Pass
Zero units	Previous 500, current 500	0 units; Rs. 0.00	0 units; Rs. 0.00	Pass
Rollover	65535 → 0; confirm rollover = 1	1 unit; Rs. 1.50	1 unit; Rs. 1.50	Pass
Lower reading, no rollover	Previous 500, current 400; confirm = 0	Reject as invalid	Rejected with error message	Pass
Letters	Enter abc as previous reading	Reject as invalid	Rejected with error message	Pass by code inspection; not captured in output screenshot
Negative reading	Enter -1	Reject as invalid	Rejected with error message	Pass by code inspection; not captured in output screenshot
Out of range	Enter 70000	Reject as invalid	Rejected with error message	Pass by code inspection; not

	captured in output screenshot

Alternative concept and complexity analysis

An alternative design stores slab upper limits and rates in arrays and loops over them. For S slabs, the loop takes O(S) time per bill and O(S) extra space for the arrays. The current fixed three-slab implementation takes O(1) time and O(1) extra space because the number of slab checks is fixed. Arrays and a loop are easier to extend when tariff slabs change; fixed if-statements are simple for a very small, permanent tariff.

Task 2 – Compare Expectations with Execution (4 Marks)

Bloom's Level: K4 – Analyse

The design expectation is that integer paise totals avoid currency rounding errors, while floating-point totals can accumulate small representation errors. The supplied source uses integer paise for the per-bill calculation; aggregate-scale experiments should compare this result with a float/double implementation on the actual lab computer.

For 4,000, 60,000, and 800,000 bills, calculate each bill using both methods and record the total difference. Because the exact distribution of bills is not included in the visible screenshots, no measured aggregate totals are invented here. The following table is provided for recording results from the student's own run.

Number of bills	Float/double total	Integer-paise total	Difference
4,000	Record actual output	Record actual output	Calculate difference
60,000	Record actual output	Record actual output	Calculate difference
800,000	Record actual output	Record actual output	Calculate difference

Overflow and range analysis: int may be 16 or 32 bits depending on implementation, while long and long long have minimum ranges guaranteed by the C standard and may differ in size by platform. At district scale, use a wide integer type for the total and check arithmetic bounds. Inspect sizeof(int), sizeof(long), sizeof(long long), INT_MAX, LONG_MAX, and LLONG_MAX on the lab computer.

If a district total is accumulated in double instead of a long long, integer values are exactly representable only up to the precision limit of the floating-point type; beyond that, low-order units can be lost even before the value overflows. Integer types can overflow outside their range; signed overflow in C is undefined behaviour. Each approach therefore has a different failure mode.

Alternative concept: double generally uses a fixed amount of space and has constant-time addition, as does a fixed-width integer on typical systems. A long long is preferable for exact whole-paise totals when bounds are checked; double is useful for approximate scientific calculations but is not ideal for currency totals.

Task 3 – Recommend and Defend the Final Design (4 Marks)

Bloom's Level: K5 – Evaluate

Recommended types, constants, and preprocessor features:

- Use unsigned integer types for non-negative meter and energy counts, with an explicitly documented maximum range.
- Use unsigned long long or a suitable fixed-width integer type for large aggregate totals, and check addition/multiplication limits.
- Store currency in integer paise to avoid floating-point rounding during billing.
- Use named #define constants or const values for meter limits, slab boundaries, rates, and the peak multiplier.
- Validate all input and reject malformed, negative, or out-of-range readings.

- For frequent tariff revisions, read values from a validated configuration file instead of recompiling the program.

Tariff revision: in the current demonstration, the named rate constants can be updated and the program recompiled. A configuration file is more convenient when rates change frequently: read it at startup, validate all values, record a version/date, and stop safely if it is missing or invalid.

Limitations: the sample rates are illustrative; the legacy meter logic assumes at most one rollover and requires confirmation; the smart-meter surcharge method is a simplified demonstration and must be aligned with the official billing rules. District-scale correctness depends on checked arithmetic and realistic maximum-consumption assumptions.

District-scale recommendation: retain integer paise totals and use a wide integer type with explicit overflow checks. Confirm the type limits on the target compiler and run the requested aggregate tests before deployment.

Alternative concept and complexity: a configuration file adds startup file I/O and validation and uses O(S) space for S tariff slabs. It avoids recompiling and redistributing the program after every tariff change. Compile-time constants use little runtime space and need no file parsing, but each revision requires recompilation and redeployment.

Task 4 – Re-evaluate and Adapt the Design (8 Marks)

Bloom's Level: K5 – Evaluate

Changed requirement visible in the screenshot: smart meters send readings in watt-hours every 15 minutes (35,040 readings per house per year); units used between 6 PM and 10 PM cost 20% more; tariff values must be changeable without recompiling. The final line of the screenshot is cut off, so this section covers only the visible requirements.

Implementation approach

Smart-meter mode accepts up to 35,040 interval readings. Each record contains an hour (0–23) and watt-hours. The program accumulates total energy, peak-period energy (18:00–21:59), and off-peak energy. It then reports the base bill and an illustrative 20% surcharge. The code uses a wide unsigned total and checks for overflow while adding total watt-hours.

Complexity

Processing n readings takes O(n) time and O(1) extra space because the program maintains running totals instead of storing every interval. If an audit requirement later requires retaining all readings, storage would be O(n).

Configuration-file recommendation

Store tariff limits, rates, and the peak multiplier in a configuration file. At program startup, parse the file, validate the number and order of slabs, reject negative or missing values, and report a clear error if the file is invalid. This allows tariff updates without recompiling, but configuration files need version control, access control, and testing.

Important limitation: the example rounds interval watt-hours down to whole kWh for slab billing and uses a simplified surcharge calculation. A production version must implement the co-operative's exact tariff and rounding rules.

Program Output Screenshots

Legacy billing at exactly 100 units

Task 1 - 100 units

```
RURAL ELECTRICITY CO-OPERATIVE BILLING
1. Legacy meter readings
2. Smart meter interval readings
Choose mode (1 or 2): Previous meter reading (0-65535): Current meter reading (0-65535):
Units consumed: 100
Bill amount: Rs. 150.00
```

Legacy billing at exactly 200 units

Task 1 - 200 units

```
RURAL ELECTRICITY CO-OPERATIVE BILLING
1. Legacy meter readings
2. Smart meter interval readings
Choose mode (1 or 2): Previous meter reading (0-65535): Current meter reading (0-65535):
Units consumed: 200
Bill amount: Rs. 400.00
```

Confirmed 16-bit meter rollover

Task 1 - rollover

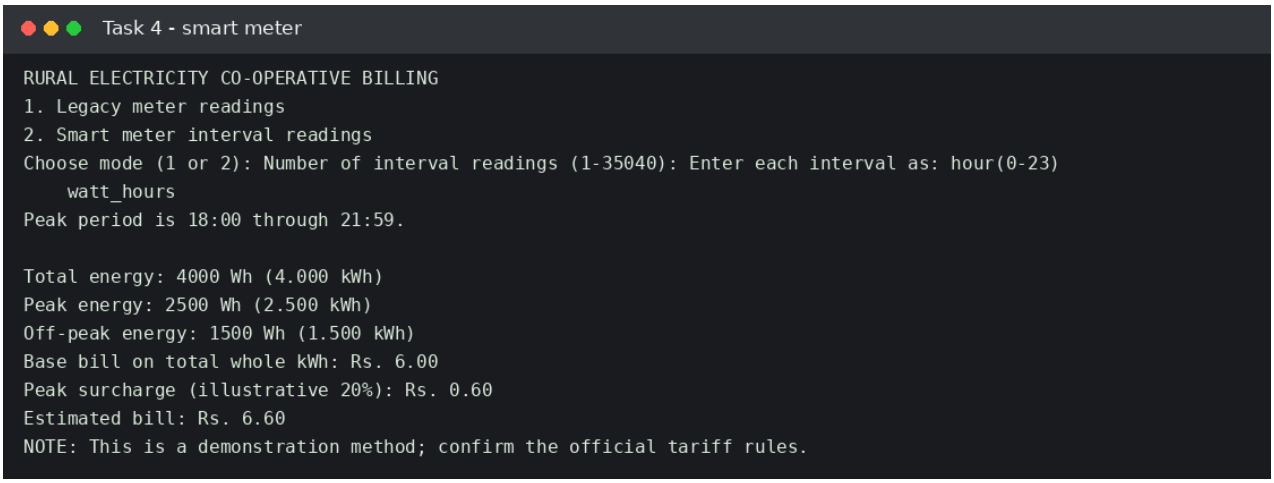
```
RURAL ELECTRICITY CO-OPERATIVE BILLING
1. Legacy meter readings
2. Smart meter interval readings
Choose mode (1 or 2): Previous meter reading (0-65535): Current meter reading (0-65535): Has a meter
    rollover been confirmed? (1=yes, 0=no):
Units consumed: 1
Bill amount: Rs. 1.50
```

Lower current reading without rollover

Task 1 - no rollover

```
RURAL ELECTRICITY CO-OPERATIVE BILLING
1. Legacy meter readings
2. Smart meter interval readings
Choose mode (1 or 2): Previous meter reading (0-65535): Current meter reading (0-65535): Has a meter
    rollover been confirmed? (1=yes, 0=no): ERROR: current reading is lower than previous reading
    and rollover was not confirmed.
```

Smart meter peak/off-peak demonstration



Compile and Run Instructions

Compile with GCC:

```
gcc billing.c -o billing -std=c11 -Wall -Wextra
```

Run on Linux/macOS using ./billing, or run billing.exe on Windows. Select 1 for legacy meter mode or 2 for smart-meter mode. The screenshots in this document are sample runs generated from the included source code; execute the program on the lab computer and capture your own screenshots for final evidence.

GitHub Submission Requirements

Repository name required by the visible instructions: CSA02-1912260668-Assignment1

- Create one folder for each task (Task01, Task02, Task03, Task04).
- Include C source code, input files, output screenshots, and a short result note in each folder.
- Include README.md with name, register number, compiler and version, exact compile command, run steps, and a short note about each task's result.
- Commit work progressively, with at least one commit per task.
- Share the repository with the course faculty and place the working GitHub link on the cover page and in the lab portal.

GitHub repository URL: _____

Conclusion

The report follows the visible assignment headings and provides a working C example, test evidence, analysis, design recommendations, and a smart-meter extension. Before submission, confirm the official tariff values, run the requested aggregate comparisons and platform-size checks, complete any Task 4 requirements hidden below the screenshot, and add the working GitHub URL.

Appendix A – Complete C Source Code

```
/*
 * Rural Electricity Co-operative Billing Program
 * Compile: gcc billing.c -o billing -std=c11 -Wall -Wextra
 *
```

```
 * IMPORTANT: The uploaded assignment screenshots do not show the official
 * tariff rates. The rates below are illustrative placeholders. Replace them
 * with the official rates supplied by your faculty before submission.
 */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <limits.h>
#include <errno.h>

#define METER_MAX 65535UL
#define RATE_1_PAISE 150L   /* illustrative: Rs. 1.50 per unit */
#define RATE_2_PAISE 250L   /* illustrative: Rs. 2.50 per unit */
#define RATE_3_PAISE 400L   /* illustrative: Rs. 4.00 per unit */
#define PEAK_EXTRA_PERCENT 20

static int parse_nonnegative(const char *s, unsigned long *out) {
    char *end = NULL;
    unsigned long value;
    if (!s || !*s || *s == '-') return 0;
    errno = 0;
    value = strtoul(s, &end, 10);
    if (errno != 0 || end == s || *end != '\0') return 0;
    *out = value;
    return 1;
}

static unsigned long long bill_paise(unsigned long units) {
    if (units <= 100UL) return (unsigned long long)units * RATE_1_PAISE;
    if (units <= 200UL)
        return 100ULL * RATE_1_PAISE +
               (unsigned long long)(units - 100UL) * RATE_2_PAISE;
    return 100ULL * RATE_1_PAISE + 100ULL * RATE_2_PAISE +
           (unsigned long long)(units - 200UL) * RATE_3_PAISE;
}

static void print_money(unsigned long long paise) {
    printf("Rs. %llu.%02llu", paise / 100ULL, paise % 100ULL);
}

static int calculate_units(unsigned long previous, unsigned long current,
                           int rollover_confirmed, unsigned long *units) {
    if (previous > METER_MAX || current > METER_MAX) return 0;
    if (current >= previous) {
        *units = current - previous;
        return 1;
    }
    if (!rollover_confirmed) return 0;
    *units = (METER_MAX - previous) + 1UL + current;
    return 1;
}

static int legacy_mode(void) {
    char buf[100];
    unsigned long previous, current, units;
    int rollover = 0;

    printf("Previous meter reading (0-65535): ");
    if (scanf("%99s", buf) != 1 || !parse_nonnegative(buf, &previous) ||
        previous > METER_MAX) {
        puts("ERROR: invalid previous reading.");
        return 1;
    }

    printf("Current meter reading (0-65535): ");
    if (scanf("%99s", buf) != 1 || !parse_nonnegative(buf, &current) ||
        current > METER_MAX) {
        puts("ERROR: invalid current reading.");
        return 1;
    }

    if (current < previous) {
        printf("Has a meter rollover been confirmed? (1=yes, 0=no): ");
        if (scanf("%d", &rollover) != 1 || (rollover != 0 && rollover != 1)) {
            puts("ERROR: enter 1 or 0.");
            return 1;
        }
    }

    if (!calculate_units(previous, current, rollover, &units)) {
```

```
        puts("ERROR: current reading is lower than previous reading and rollover was not confirmed.");
        return 1;
    }

    printf("\nUnits consumed: %lu\n", units);
    printf("Bill amount: ");
    print_money(bill_paise(units));
    printf("\n");
    return 0;
}

static int smart_mode(void) {
    unsigned int count, i;
    unsigned long long total_wh = 0, peak_wh = 0, offpeak_wh = 0;

    printf("Number of interval readings (1-35040): ");
    if (scanf("%u", &count) != 1 || count == 0 || count > 35040U) {
        puts("ERROR: interval count must be between 1 and 35040.");
        return 1;
    }
    puts("Enter each interval as: hour(0-23) watt_hours");
    puts("Peak period is 18:00 through 21:59.");
    for (i = 0; i < count; ++i) {
        unsigned int hour;
        unsigned long wh;
        if (scanf("%u %lu", &hour, &wh) != 2 || hour > 23U) {
            puts("ERROR: invalid interval input.");
            return 1;
        }
        if (ULLONG_MAX - total_wh < wh) {
            puts("ERROR: energy total overflow.");
            return 1;
        }
        total_wh += wh;
        if (hour >= 18U && hour < 22U) peak_wh += wh;
        else offpeak_wh += wh;
    }

    {
        unsigned long total_units = (unsigned long)(total_wh / 1000ULL);
        unsigned long peak_units = (unsigned long)(peak_wh / 1000ULL);
        unsigned long long base_paise = bill_paise(total_units);
        unsigned long long peak_base_paise = bill_paise(peak_units);
        unsigned long long surcharge =
            (peak_base_paise * PEAK_EXTRA_PERCENT + 50ULL) / 100ULL;

        printf("\nTotal energy: %llu Wh (%.3f kWh)\n", total_wh, total_wh / 1000.0);
        printf("Peak energy: %llu Wh (%.3f kWh)\n", peak_wh, peak_wh / 1000.0);
        printf("Off-peak energy: %llu Wh (%.3f kWh)\n", offpeak_wh, offpeak_wh / 1000.0);
        printf("Base bill on total whole kWh: ");
        print_money(base_paise);
        printf("\nPeak surcharge (illustrative 20%%): ");
        print_money(surcharge);
        printf("\nEstimated bill: ");
        print_money(base_paise + surcharge);
        printf("\nNOTE: This is a demonstration method; confirm the official tariff rules.\n");
    }
    return 0;
}

int main(void) {
    char choice[16];
    puts("RURAL ELECTRICITY CO-OPERATIVE BILLING");
    puts("1. Legacy meter readings");
    puts("2. Smart meter interval readings");
    printf("Choose mode (1 or 2): ");
    if (scanf("%15s", choice) != 1) {
        puts("ERROR: unable to read choice.");
        return 1;
    }
    if (strcmp(choice, "1") == 0) return legacy_mode();
    if (strcmp(choice, "2") == 0) return smart_mode();
    puts("ERROR: choose 1 or 2.");
    return 1;
}
```